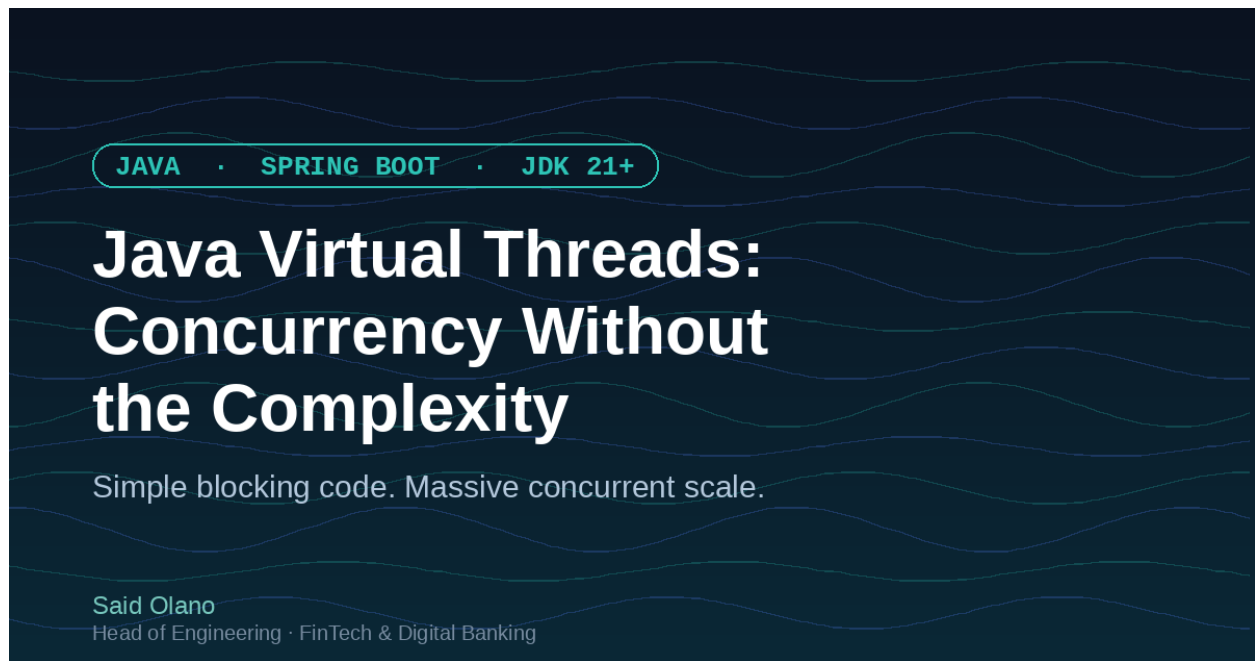




Java Virtual Threads: Concurrency Without the Complexity

By Said Olano - Head of Engineering, FinTech & Digital Banking

If you've built Java services for a while, you know the trade-off by heart: thread-per-request is simple but doesn't scale past a few thousand concurrent connections, and reactive programming (Project Reactor, Mono/Flux, callback chains) scales beautifully but turns your codebase into something only half the team can debug.

JDK 21 gives us a third option: Virtual Threads, delivered through Project Loom. They let you keep writing simple, blocking-style Java, while the JVM handles the scheduling gymnastics that used to require a reactive framework.

The problem with traditional concurrency

A platform (OS) thread is expensive. Each one reserves around 1MB of stack space and involves the operating system in every context switch. Spin up a thread per incoming request and a server tips over somewhere between a few thousand and ten thousand concurrent connections — long before the CPU itself is the bottleneck.

The traditional fix has been asynchronous, non-blocking code: reactive streams, callbacks, `CompletableFuture` chains. They work, but they change how you write and read code, how you debug stack traces, and how you reason about failures.

Enter virtual threads

A virtual thread is a lightweight thread managed by the JVM instead of the OS. You can create millions of them, because they don't map 1:1 to OS threads — the JVM multiplexes many virtual threads onto a small pool of "carrier" threads, parking a virtual thread whenever it blocks on I/O and freeing the carrier thread to run someone else.

The result: you write ordinary blocking code —

```
public String fetchUser(String id) {  
    var response = httpClient.send(request, BodyHandlers.ofString());  
    return response.body();  
}
```

— and the runtime gives you the throughput characteristics of async code, without the mental overhead.

Turning it on in Spring Boot

As of Spring Boot 3.2, enabling virtual threads for your web server and task executors is a one-line change:

```
spring.threads.virtual.enabled=true
```

That's it. Your `@RestController` endpoints, JDBC calls (with a JDBC 4.3+ driver), and `RestClient`/`RestTemplate` calls now run on virtual threads by default. No code rewrite, no reactive types leaking into your service layer.

Where they shine

Virtual threads are built for I/O-bound workloads: REST APIs calling downstream services, database queries, file access, anything that spends most of its time waiting rather than computing. This is exactly the profile of most Spring Boot microservices.

Where they don't help

They're not a silver bullet for CPU-bound work. Running a virtual thread doesn't make matrix multiplication or JSON serialization faster — you still have the same number of CPU cores. If your service is compute-heavy, virtual threads won't move the needle much.

The gotcha: thread pinning

Virtual threads can get "pinned" to their carrier thread in a couple of situations — most notably inside a synchronized block or method that blocks. While pinned, the carrier thread can't be reused, which defeats the purpose. The fix is usually straightforward: replace `synchronized` with `java.util.concurrent.locks.ReentrantLock` in hot paths that run on virtual threads. (JDK 24 removes most of this limitation, but if you're on 21 LTS, keep it in mind.)

Wrapping up

Virtual threads are one of the rare JVM features that give you a real scalability win for the cost of a config flag and a little awareness of the edge cases. If you're running Spring Boot on JDK 21+, it's

worth testing in a staging environment before your next load test.

Have you adopted virtual threads in a production Spring Boot service? I'd love to hear what you ran into — drop a comment below.